ST-ROBOT · TÀI LIỆU TỰ HỌC

VEXcode VR

Học thuật toán trên robot ảo; không cần bộ kit VEX IQ để bắt đầu.

6 bài học · Lý thuyết, ví dụ, thực hành, kiểm tra và tài liệu tham khảo

Lộ trình học tập

1. Chuỗi lệnh và hướng
2. Vòng lặp và hình vuông
3. Tránh vật cản có điều kiện
4. Thử thách: giao hàng ảo
5. Tìm đường trên lưới bằng BFS
6. Thực nghiệm: so sánh hai thuật toán

Cách sử dụng tài liệu

Đọc mục tiêu và kiến thức nền; tự dự đoán trước ví dụ; thực hành theo các bước; ghi lại dữ liệu và giải thích sai lệch. Sau khi tự làm, đối chiếu đáp án và tự kiểm tra. Phòng lab bên ngoài không tự đồng bộ điểm; hoạt động mô phỏng không thay thế kiểm tra an toàn phần cứng thật.

Bài 1/6 · Chuỗi lệnh và hướng

Dự đoán và thử chuỗi tiến–rẽ ngắn để đưa robot đến một ô đích, ghi được vị trí và hướng sau mỗi lệnh.

Chuẩn bị

Chưa cần viết Python hoặc có robot. Biết mũi tên đang chỉ hướng nào. Trên lưới: F tiến một ô, R rẽ phải 90° tại chỗ, L rẽ trái 90° tại chỗ; Reset đưa robot về (0,0), nhìn sang phải (đông).

Sau bài học, bạn có thể

1. Dự đoán rồi kiểm tra một đường đi có lệnh tiến và rẽ.

2. Ghi đủ vị trí và hướng; đếm riêng số lệnh với số ô đã tiến.

3. Nhận biết lưới nội bộ, GearsBot và VEXcode VR có quy ước, đơn vị riêng.

Cùng hiểu bài từ đầu

Trước khi đọc một chương trình dài, hãy học cách làm robot bằng mũi tên trên giấy. Lệnh tiến đổi ô, lệnh rẽ đổi hướng. Kết quả của một lệnh phụ thuộc trạng thái mà lệnh trước để lại, nên thứ tự là một phần của lời giải.

Đứng ở một ngã tư, "đi thẳng rồi rẽ phải" và "rẽ phải rồi đi thẳng" không đưa bạn đến cùng một chỗ. Hướng nhìn trước khi bước quyết định bước ấy đi về đâu.

Robot trên lưới cũng thực hiện từng lệnh theo thứ tự. Muốn dự đoán, lưu cả tọa độ và hướng sau mỗi lệnh thay vì đếm tổng số F rồi tổng số R.

Giới hạn của ví von: Bàn lưới là mô hình rời rạc dùng F/R/L của bài. VEXcode VR có Playground, hệ tọa độ và đơn vị API riêng; một ô ở đây không tự tương ứng với một quãng đường VR.

Những từ cần hiểu

1. Chuỗi lệnh: Danh sách hành động được thực hiện theo thứ tự; đổi thứ tự có thể đổi kết quả.

2. Trạng thái · (x,y,hướng): Thông tin cần giữ để tính bước tiếp theo. Bài dùng x tăng sang phải, y tăng xuống dưới.

3. F · tiến một ô: Đổi tọa độ theo hướng đang nhìn, giữ hướng. Khi hướng nam, F tăng y.

4. R và L · rẽ tại chỗ: Đổi hướng 90° phải hoặc trái và giữ ô hiện tại trong mô hình.

Vì sao F,R khác R,F?

Bắt đầu với: Cùng bắt đầu tại (1,1,đông) trong vùng lưới cho phép. So sánh chuỗi A = F,R và chuỗi B = R,F.

Chạy A bằng hai trạng thái

F đầu tiên đưa robot đến (2,1,đông). R sau đó đổi hướng thành nam, nên kết thúc (2,1,nam).

Đặt lại đầu vào rồi chạy B

B phải bắt đầu lại ở (1,1,đông). R đầu tiên cho (1,1,nam); F sau đó tăng y nên tới (1,2,nam).

Đối chiếu điều thực sự khác

A và B đều có một F, một R và cùng hướng cuối nam. Nhưng vị trí cuối khác nhau vì F được thực hiện khi robot hướng khác nhau.

Kết quả: A kết thúc ở (2,1,nam); B ở (1,2,nam). Số lượng từng loại lệnh giống nhau không bảo đảm trạng thái cuối giống nhau.

Dòng bảng trạng thái nối đầu ra của lệnh trước thành đầu vào của lệnh sau. Đây là cách kiểm tra thuật toán có thể dùng trước khi chạy mô phỏng.

Tự giải thích rồi đối chiếu

Một R có làm robot tiến thêm một ô không?

Không. Trong mô hình này R chỉ đổi hướng; phải có F mới dịch chuyển sang ô tiếp theo.

Nếu hai tuyến cùng đích nhưng khác hướng cuối, có coi là cùng trạng thái không?

Không. Trạng thái bao gồm hướng. Chúng chỉ cùng đáp ứng bài toán nếu yêu cầu đích cho phép mọi hướng cuối.

Chọn độ sâu phù hợp

Mới bắt đầu · Dùng mũi tên thay cho ký hiệu khó nhớ

Vẽ mũi tên tại ô đầu. Với F, trượt nó một ô; với R/L, xoay nó ngay tại chỗ. Ghi lại sau mỗi hành động để giảm việc phải nhớ trong đầu.

Tự dự đoán F,R,F từ (0,0,đông), rồi mới đối chiếu trong Lab.

Đã học cơ bản · Đọc trạng thái như dữ liệu

Mỗi lệnh nhận một trạng thái và trả trạng thái mới. Dùng quy ước x/y của bài nhất quán; đừng mặc định y tăng lên trên như một hệ tọa độ toán học khác.

Viết bảng cho F,R,F,L,F và kiểm tra riêng vị trí, hướng, số bước tiến.

Muốn tìm hiểu sâu · Tách mô hình lưới và API chuyển động

Một lệnh drive_for trong VR có thể cần đơn vị mm hoặc inch, còn rẽ có đơn vị góc và quy ước nền tảng. Cần cấu hình đúng sân và đọc API trước khi chuyển kế hoạch lưới thành chương trình; GearsBot chạy trên hệ riêng.

Nêu những thông tin cần bổ sung để biến F thành một lệnh quãng đường có nghĩa trong Playground đã chọn.

Kiến thức trọng tâm

Việc đầu tiên: tiến và rẽ là hai việc khác nhau

Hãy coi robot như một mũi tên trên giấy. Tiến là đi một ô theo mũi tên; rẽ phải hoặc trái là xoay mũi tên mà vẫn ở ô hiện tại. Muốn biết lệnh tiến tiếp theo đưa robot đi đâu, hãy nhìn hướng của mũi tên ngay lúc đó. Làm thử một đường ngắn trước khi đọc mã điều khiển motor.

Đọc thêm · Chuỗi lệnh được thực hiện theo thứ tự

Mỗi lệnh nhận trạng thái hiện tại và tạo trạng thái tiếp theo. Lệnh tiến đổi vị trí theo hướng đang quay; lệnh rẽ tại chỗ đổi hướng. Thay thứ tự lệnh thường thay kết quả: tiến rồi rẽ khác rẽ rồi tiến. Muốn dự đoán đúng phải ghi đủ (x,y,hướng), không chỉ tọa độ.

Đọc thêm · Quy ước của bàn lưới trong bài

Bàn lưới dùng x tăng sang phải, y tăng xuống dưới; trạng thái đầu (0,0,đông). F là tiến một ô, R là rẽ phải 90°, L là rẽ trái 90°. Ví dụ hướng nam thì F tăng y; hướng bắc thì F giảm y. Chỉ dùng quy ước này cho bàn lưới, không áp đặt cho mọi simulator.

Đọc thêm · Chuyển sang VEXcode VR

VEXcode VR có các Playground và đơn vị đo của riêng nó; đọc chỉ dẫn sân và chọn đúng robot/sân trước khi chạy. Lệnh quãng đường có thể dùng mm hoặc inch theo API đã chọn; một ô trong bài không tự bằng một khoảng cách VR. GearsBot là nền tảng bổ trợ riêng, không phải VEXcode VR và không chạy nguyên API VR.

Ví dụ có lời giải

Từ (0,0,đông): F,F,F đưa robot đến (3,0,đông); R cho (3,0,nam); F,F cho (3,2,nam). Chuỗi có 6 hành động nhưng chỉ 5 bước tiến; hành động rẽ không cộng một ô quãng đường.

Lỗi thường gặp và cách tránh

1. Lệnh tiến không phải luôn tăng x; nó phụ thuộc hướng hiện tại.

2. Ô lưới, mm của sân VR và vị trí thực của robot là các đại lượng khác nhau.

Các bước thực hành

1. Dự đoán: từ (0,0,đông), vẽ đường đi F,F,R,F. Ghi mũi tên hướng robot sau mỗi lệnh; x tăng sang phải, y tăng xuống.

2. Thao tác: vào Lab → Bàn thí nghiệm, Reset rồi nhập F,F,R,F và chạy mô hình lưới.

3. Quan sát: đối chiếu từng trạng thái với hình vẽ. Khoanh hàng R để kiểm tra rằng vị trí không đổi nhưng hướng đã đổi.

4. Giải thích: ghi số lệnh đã làm và số ô tiến; nêu vì sao hai số khác nhau.

5. Thử tiếp: dự đoán chuỗi R,F,F,F, Reset rồi chạy. So sánh hai đích để giải thích tác dụng của thứ tự lệnh.

6. Mở rộng: dùng hướng dẫn Lab để chạy mẫu Python trong GearsBot, hoặc chuyển thuật toán sang VEXcode VR. Kiểm tra robot, sân và đơn vị của từng nền tảng trước khi so sánh.

Mã mẫu · Python · GearsBot (ev3dev2)

```
from math import pi, sqrt
from time import time, sleep
from ev3dev2.motor import MoveTank, OUTPUT_A, OUTPUT_B
from ev3dev2.sensor import INPUT_4
from ev3dev2.sensor.virtual import GPSSensor

tank = MoveTank(OUTPUT_A, OUTPUT_B)
gps = GPSSensor(INPUT_4)
WHEEL_DIAMETER_MM = 56.0
TRACK_WIDTH_MM = 152.0
SPEED = 25

def move_wheels(left_speed, right_speed, degrees, timeout_s=8.0):
  start_left = tank.left_motor.position
  start_right = tank.right_motor.position
  deadline = time() + timeout_s
  tank.on_for_degrees(left_speed, right_speed, degrees,
              brake=True, block=False)
  try:
    sleep(0.05)
    while time() < deadline:
      if tank.left_motor.is_stalled or tank.right_motor.is_stalled:
        raise RuntimeError("Motor stalled; check the course")
      if not (tank.left_motor.is_running or tank.right_motor.is_running):
        left_done = abs(tank.left_motor.position - start_left)
        right_done = abs(tank.right_motor.position - start_right)
        if min(left_done, right_done) >= degrees - 5:
          return
      sleep(0.02)
    raise RuntimeError("Motion timeout; reset before retrying")
  finally:
    tank.off(brake=True)

def forward(distance_mm):
  degrees = 360.0 * distance_mm / (pi * WHEEL_DIAMETER_MM)
  move_wheels(SPEED, SPEED, degrees)

def turn_right(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(SPEED, -SPEED, degrees)

def turn_left(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(-SPEED, SPEED, degrees)

# Initial logical heading: east. One cell is 100 mm.
x, y, heading = 0, 0, 0
vectors = [(1, 0), (0, 1), (-1, 0), (0, -1)]
names = ["east", "south", "west", "north"]
try:
  for command in "F F R F".split():
    if command == "F":
      forward(100)
      dx, dy = vectors[heading]
      x, y = x + dx, y + dy
    elif command == "R":
      turn_right()
```

```
        heading = (heading + 1) % 4
      elif command == "L":
        turn_left()
        heading = (heading - 1) % 4
      else:
        raise ValueError("Unknown command")
      print(command, "->", x, y, names[heading])
  print("GPS endpoint (cm):", gps.position[:2])
finally:
  tank.off(brake=True)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn robot Single Sensor Line Follower (singleFollower), mở Python và dán toàn bộ chương trình. Robot dùng bánh trái A, bánh phải B; đường kính bánh 56 mm, khoảng cách hai bánh 152 mm. Chọn sân rộng, phẳng; Reset trước mỗi lượt rồi Run. Chuỗi F F R F chạy trực tiếp với một ô bằng 100 mm. Console ghi vị trí và hướng theo lưới sau mỗi lệnh; hướng đông quy ước trùng hướng xuất phát của robot. Đổi chuỗi thành F R F L F để làm bài tập, Reset rồi chạy. Tọa độ logic giúp kiểm tra thứ tự lệnh; GPS cung cấp số đo riêng trên sân GearsBot.

Bài tập áp dụng

Dự đoán bằng hình mũi tên hoặc bảng trạng thái chuỗi F,R,F,L,F từ (0,0,đông). Reset, chạy bàn lưới và ghi một điểm khớp hoặc sai với dự đoán. Đếm riêng số lệnh và số ô đã tiến; phần thêm: thử bắt đầu hướng nam trên giấy.

Gợi ý đối chiếu sau khi tự làm

Từ hướng đông: (1,0,đông), (1,0,nam), (1,1,nam), (1,1,đông), (2,1,đông). Có 5 lệnh, gồm 3 bước tiến và 2 lần rẽ tại chỗ. Nếu xuất phát hướng nam, trên lưới không giới hạn đích là (−1,2,nam); bàn có biên có thể chặn chuyển động nên cần kiểm tra phạm vi trước khi chạy.

Tự kiểm tra mức hiểu

1. Trong F,F,R,F, lệnh nào đổi hướng nhưng giữ vị trí?

2. Vì sao đổi thứ tự thành R,F,F,F làm thay đổi đích?

Câu hỏi trắc nghiệm

Lệnh rẽ tại chỗ trong mô hình thay đổi...

1. Hướng

2. Số ô đã đi

3. Kích thước sân

Đáp án: 1. Rẽ tại chỗ đổi hướng của robot. Trong mô hình lý tưởng, tâm robot chưa dịch chuyển sang ô khác nên không tăng số ô đã đi.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEXcode VR — môi trường lập trình chính thức

https://vr.vex.com/

VEXcode VR — tài liệu Python chính thức

https://api.vex.com/vr/home/python/

VEX CS Level 1 — bài học VEXcode VR

https://education.vex.com/stemlabs/cs/cs-level-1-vexcode-vr

GEARS — cấu hình Single Sensor Line Follower

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GEARS — API ev3dev2.motor trong simulator

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GEARS — API GPS và đơn vị tọa độ

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/virtual.py

Bài 2/6 · Vòng lặp và hình vuông

Nhận ra nhóm lệnh lặp và xác định đúng phần thân của vòng lặp.

Chuẩn bị

Hoàn thành Chuỗi lệnh và hướng. Biết 4 × 90° = 360° và phân biệt một hành động rẽ với một bước tiến.

Sau bài học, bạn có thể

1. Nhận ra nhóm lệnh lặp và xác định đúng phần thân của vòng lặp.

2. Tính tổng số hành động, bước tiến và góc quay của chương trình lặp.

3. Kiểm tra cả vị trí cuối lẫn hướng cuối để xác nhận một đường hình vuông.

Cùng hiểu bài từ đầu

Khi một nhóm hành động xuất hiện nhiều lần, bạn có thể mô tả nhóm đó một lần rồi yêu cầu lặp. Điều khó nhất thường là chọn đúng phần nằm trong vòng lặp: lặp cả "tiến rồi rẽ" khác với chỉ lặp "tiến".

Một động tác thể dục gồm bước sang bên rồi xoay người. Làm cả động tác bốn lần khác với bước bốn lần rồi chỉ xoay một lần.

Thân vòng lặp là trọn nhóm hành động được làm lại. REPEAT 4 {F; R} chạy F,R,F,R,F,R,F,R theo đúng thứ tự ấy.

Giới hạn của ví von: Một động tác người có thể không chính xác bằng một ô hoặc một góc. Vòng lặp chỉ rút gọn mô tả; nó không tự kiểm tra biên sân hay sửa chuyển động sai.

Những từ cần hiểu

1. Thân vòng lặp: Nhóm lệnh nằm bên trong Repeat hoặc cùng khối thụt lề Python; toàn bộ nhóm được lặp.

2. Số lần lặp: Số lần thực hiện cả thân, không phải tổng số lệnh bên trong.

3. Số hành động và quãng đường: Rẽ là hành động nhưng không thêm quãng đường lưới. Cần đếm riêng F và R/L.

4. Đường kín: Đường kết thúc ở vị trí đầu. Muốn cùng trạng thái đầu còn phải kiểm tra cả hướng cuối.

Theo từng lượt lặp của một hình vuông cạnh hai ô

Bắt đầu với: Bắt đầu (0,0,đông), vùng lưới đủ cho x,y từ 0 đến 2. Chạy REPEAT 4 {F; F; R}.

Hiểu một lần làm cả thân

Lần 1 có hai F rồi một R: tới (2,0,nam). Không tách hai F và R thành các nhóm lặp độc lập.

Theo dõi hai lần tiếp theo

Lần 2 từ hướng nam tới (2,2,tây). Lần 3 từ hướng tây tới (0,2,bắc). Hướng cuối mỗi lượt là hướng dùng cho F đầu lượt sau.

Khép kín và đếm đúng

Lần 4 đưa về (0,0,đông). Có 4 × 2 = 8 bước tiến và 4 lần rẽ, tổng 12 hành động. Chu vi trên mô hình là 8 ô, không phải 12 ô.

Kết quả: Robot trở về cả vị trí và hướng đầu, đi 8 ô và thực hiện 4 lần rẽ. Bốn lượt lặp không đồng nghĩa với bốn hành động.

Theo dõi sau mỗi thân vòng lặp tạo bốn mốc kiểm tra ngắn. Nếu đường không khép kín, đối chiếu xem thân thiếu F/R hay chương trình đi ra ngoài vùng cho phép.

Tự giải thích rồi đối chiếu

REPEAT 4 {F}; R có giống REPEAT 4 {F; R} không?

Không. Cách đầu tiến bốn ô rồi rẽ một lần. Cách sau đan xen tiến và rẽ bốn lần, thường tạo đường khác.

Bỏ R cuối mà vẫn về ô đầu, có cùng trạng thái đầu không?

Không. Vị trí có thể đúng nhưng hướng còn khác. Cần kiểm tra yêu cầu có bao gồm hướng cuối hay không.

Chọn độ sâu phù hợp

Mới bắt đầu · Khoanh trọn nhóm được lặp

Viết F,F,R lên ba thẻ, khoanh chúng thành một nhóm và chép cả nhóm bốn lần. Nhìn bản trải dài trước khi dùng Repeat.

Chỉ ra các lệnh thuộc thân trên bản khối hoặc qua thụt lề Python.

Đã học cơ bản · Dùng số lần lặp để đếm

Nhân riêng số F trong thân với số lượt, rồi làm tương tự với R. Chỉ dùng số F cho quãng đường ô, không dùng tổng số hành động.

Thiết kế hình vuông cạnh 3 ô và tính số bước tiến, số rẽ trước khi chạy.

Muốn tìm hiểu sâu · Xác định điều kiện để đường khép kín

Bốn góc quay ngoài 90° cho tổng 360°, nhưng tổng góc quay đủ chưa chứng minh vị trí trở lại đầu nếu độ dài cạnh khác nhau. Cần xét cả chuyển vị và hướng, đồng thời mọi trạng thái trung gian phải nằm trong sân hợp lệ.

Nêu một chuỗi có tổng góc rẽ 360° nhưng không trở lại vị trí đầu; giải thích điều kiện còn thiếu.

Kiến thức trọng tâm

Lặp lại một nhóm lệnh có cấu trúc

REPEAT 4 {F; R} nghĩa là chạy cả nhóm F rồi R bốn lần, không phải chạy F bốn lần rồi R một lần. Trong editor khối, những khối nằm bên trong Repeat tạo thành thân vòng lặp. Trong Python, thụt lề cho biết câu lệnh nào nằm trong thân; sai thụt lề làm thay đổi thuật toán.

Đếm theo từng loại hành động

Nếu thân có một F và một R, số hành động là 4 × 2 = 8; số bước tiến là 4; số rẽ là 4. Quãng đường lưới là 4 ô, không phải 8 ô. Với cạnh dài a đơn vị trên mô hình lý tưởng, chu vi hình vuông 4a; bốn góc quay ngoài 90° cho tổng 360°.

Xác minh một đường kín

Từ (0,0,đông), lặp F,R bốn lần về cùng vị trí và cùng hướng trên lưới không bị giới hạn. Bàn hiện tại có biên: một đường phía ngoài biên sẽ không được coi là hoàn thành đúng. Với simulator liên tục, phải kiểm tra đơn vị cạnh và góc, không chỉ đếm số lần Repeat.

Ví dụ có lời giải

Lặp bốn lần F,R: sau mỗi nhóm là (1,0,nam), (1,1,tây), (0,1,bắc), (0,0,đông). Nếu bỏ R cuối thì vẫn ở (0,0) nhưng hướng bắc; vị trí đúng chưa đủ để khẳng định cùng trạng thái cuối.

Lỗi thường gặp và cách tránh

1. Đếm 8 hành động không có nghĩa robot tiến 8 ô.

2. Về cùng điểm đầu nhưng khác hướng không phải cùng trạng thái đầu.

Các bước thực hành

1. Vẽ bốn cạnh và ghi trạng thái sau từng nhóm F,R. Tính trước số tiến, số rẽ và tổng hành động.

2. Trong bàn lưới, nhập chuỗi triển khai F,R bốn lần vì ô lệnh nội bộ dùng lệnh đơn. Reset rồi chạy để đối chiếu.

3. Trong nền tảng có khối Repeat, đặt cả tiến và rẽ vào bên trong Repeat 4; so sánh với bản tiến lặp 4 rồi mới rẽ.

4. Thử thiếu lệnh rẽ cuối hoặc đổi số lần lặp thành 3. Báo cáo riêng vị trí và hướng cuối, không chỉ hình vẽ đường đi.

Mã mẫu · Python · GearsBot (ev3dev2)

```
from math import pi, sqrt
from time import time, sleep
from ev3dev2.motor import MoveTank, OUTPUT_A, OUTPUT_B
from ev3dev2.sensor import INPUT_4
from ev3dev2.sensor.virtual import GPSSensor

tank = MoveTank(OUTPUT_A, OUTPUT_B)
gps = GPSSensor(INPUT_4)
WHEEL_DIAMETER_MM = 56.0
TRACK_WIDTH_MM = 152.0
SPEED = 25

def move_wheels(left_speed, right_speed, degrees, timeout_s=8.0):
  start_left = tank.left_motor.position
  start_right = tank.right_motor.position
  deadline = time() + timeout_s
  tank.on_for_degrees(left_speed, right_speed, degrees,
            brake=True, block=False)
  try:
    sleep(0.05)
    while time() < deadline:
      if tank.left_motor.is_stalled or tank.right_motor.is_stalled:
        raise RuntimeError("Motor stalled; check the course")
      if not (tank.left_motor.is_running or tank.right_motor.is_running):
        left_done = abs(tank.left_motor.position - start_left)
        right_done = abs(tank.right_motor.position - start_right)
        if min(left_done, right_done) >= degrees - 5:
          return
      sleep(0.02)
    raise RuntimeError("Motion timeout; reset before retrying")
  finally:
    tank.off(brake=True)

def forward(distance_mm):
  degrees = 360.0 * distance_mm / (pi * WHEEL_DIAMETER_MM)
  move_wheels(SPEED, SPEED, degrees)

def turn_right(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(SPEED, -SPEED, degrees)

def turn_left(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(-SPEED, SPEED, degrees)

start = gps.position
try:
  for side in range(4):
    forward(200)
    turn_right(90)
    print("Completed side:", side + 1, "GPS (cm):", gps.position[:2])
  end = gps.position
  error_mm = 10 * sqrt((end[0] - start[0]) ** 2
            + (end[1] - start[1]) ** 2)
  print("Return-position error (mm):", round(error_mm, 1))
  print("Commands: 4 forward, 4 right turns")
finally:
```

```
tank.off(brake=True)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn robot Single Sensor Line Follower (singleFollower), mở Python và dán toàn bộ chương trình. Robot dùng bánh trái A, bánh phải B; đường kính bánh 56 mm, khoảng cách hai bánh 152 mm. Chọn sân rộng, phẳng; Reset trước mỗi lượt rồi Run. Vòng lặp chạy bốn cạnh 200 mm, mỗi cạnh kết thúc bằng một lần quay phải 90°. Console xuất vị trí sau từng cạnh và sai lệch vị trí cuối so với đầu bằng mm. Chọn sân đủ chỗ cho hình vuông. Để so sánh bài tập B, đưa turn_right(90) ra ngoài vòng for, Reset rồi chạy lại; quan sát cả vị trí lẫn hướng cuối.

Bài tập áp dụng

So sánh A = REPEAT 4 {F,R} với B = REPEAT 4 {F}; R từ cùng trạng thái đầu. Đếm số tiến/rẽ/hành động và dự đoán vị trí/hướng trong lưới đủ rộng.

Gợi ý đối chiếu sau khi tự làm

A có 4 tiến + 4 rẽ = 8 hành động, về (0,0,đông). B có 4 tiến + 1 rẽ = 5 hành động, tới (4,0,nam) nếu lưới không chặn. Dấu ngoặc/thụt lề quyết định phần lặp, không phải chỉ từ “4 lần”.

Tự kiểm tra mức hiểu

1. Chỉ được đúng các câu lệnh nằm trong thân Repeat.

2. Tính đúng 4 tiến, 4 rẽ, 8 hành động của hình vuông.

3. Nêu được tác động của việc bỏ lần rẽ cuối đối với vị trí và hướng.

Câu hỏi trắc nghiệm

Lặp bốn lần nhóm hai lệnh tạo bao nhiêu hành động?

1. 4

2. 6

3. 8

Đáp án: 3. Một nhóm có hai hành động và được lặp bốn lần, nên có 4 × 2 = 8 hành động. Chỉ bốn trong số đó là bước tiến; bốn hành động còn lại đổi hướng.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEXcode VR — môi trường lập trình chính thức

https://vr.vex.com/

VEXcode VR — tài liệu Python chính thức

https://api.vex.com/vr/home/python/

VEX CS Level 1 — bài học VEXcode VR

https://education.vex.com/stemlabs/cs/cs-level-1-vexcode-vr

GEARS — cấu hình Single Sensor Line Follower

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GEARS — API ev3dev2.motor trong simulator

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GEARS — API GPS và đơn vị tọa độ

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/virtual.py

Bài 3/6 · Tránh vật cản có điều kiện

Thiết kế quy tắc tránh vật cản có ngưỡng, dữ liệu hợp lệ và điều kiện dừng.

Chuẩn bị

Biết vòng lặp, câu điều kiện và dấu <. Hiểu bàn cảm biến chỉ so sánh số và bàn lưới có quy ước riêng.

Sau bài học, bạn có thể

1. Thiết kế quy tắc tránh vật cản có ngưỡng, dữ liệu hợp lệ và điều kiện dừng.

2. Kiểm thử dưới/bằng/trên ngưỡng và một tình huống ngõ cụt hoặc vòng lặp.

3. Phân biệt phản ứng cục bộ với thuật toán tìm đường tới đích.

Cùng hiểu bài từ đầu

Một quy tắc tránh vật cản có thể ngắn mà vẫn có nhiều trường hợp cần nghĩ tới. Bạn sẽ kiểm tra số đo, chọn nhánh rồi đọc lại sau khi rẽ. Cách này giúp robot phản ứng với phía trước, nhưng chưa đủ để hứa tìm được mọi đích.

Trong phòng lạ, bạn nhìn phía trước; thấy bị chắn thì đứng lại, quay người rồi nhìn lại. Quay sang phải không tự bảo đảm hướng mới đã thông.

Robot cần một vòng đọc → quyết định → hành động → đọc lại. Nếu chỉ rẽ rồi tiến ngay, nó có thể đi vào một vật khác ở phía vừa quay sang.

Giới hạn của ví von: Một người có thể nhớ sơ đồ phòng; quy tắc một cảm biến trong bài không có bản đồ hay kế hoạch toàn cục. Nó có thể lặp trong ngõ cụt dù từng lần đều làm đúng quy tắc.

Những từ cần hiểu

1. Điều kiện cục bộ: Quyết định dựa trên thông tin gần robot hiện tại, ví dụ khoảng cách phía trước.

2. Ngưỡng · T: Ranh giới cùng đơn vị với d. Bài dùng d < 20 cm để chọn dừng rồi rẽ.

3. Đọc lại sau hành động: Lấy thông tin mới sau khi vị trí hoặc hướng thay đổi; không dùng mẫu cũ như mô tả mãi môi trường mới.

4. Giới hạn chạy: Số bước hoặc thời gian tối đa để kết thúc cả khi chưa tới đích; cần báo rõ chưa hoàn thành.

Rẽ một lần chưa đủ để chọn tiến

Bắt đầu với: T = 20 cm, dữ liệu đều dùng cm. Robot lần lượt đo phía trước: 15 cm; sau rẽ đo lại 12 cm; sau lần rẽ nữa đo được 30 cm. Giữ giới hạn bước và điều kiện tới đích đã đặt.

Đọc mẫu đầu và chọn nhánh

15 < 20 nên dừng rồi rẽ phải. Số 15 mô tả hướng cũ; chưa cho biết hướng mới có trống không.

Đọc lại trước khi đi

Sau rẽ đo 12 < 20 nên vẫn chưa tiến; tiếp tục dừng và xử lý rẽ theo quy tắc. Không coi lần rẽ trước như bảo đảm tránh va chạm.

Dùng mẫu mới để cho phép tiến

Sau lần rẽ tiếp, 30 < 20 sai nên chọn tiến chậm. Tiếp tục kiểm tra theo vòng lặp, không mở quyền tiến mãi từ một mẫu 30 cm.

Kết quả: Nhánh được chọn là dừng–rẽ, dừng–rẽ, tiến chậm. Nếu mất số đo ở bất cứ bước nào, dừng và báo lỗi theo yêu cầu thay vì suy ra phía trước trống.

Nhật ký khoảng cách và nhánh giải thích vì sao robot hành động. Đến giới hạn mà chưa tới đích thì báo chưa hoàn thành; tránh được vài vật chưa đồng nghĩa tìm đường thành công.

Tự giải thích rồi đối chiếu

Ở đúng 20 cm, quy tắc d < 20 chọn nhánh nào?

Nhánh ngược lại, tức tiến chậm theo đặc tả bài. Dùng ≤ sẽ đổi hành vi tại biên và phải kiểm thử lại.

Nếu robot quay vòng trong một ngõ cụt, có chắc phép so sánh viết sai không?

Không. Đó có thể là giới hạn của chiến lược chỉ nhìn cục bộ và luôn rẽ một phía; cần đọc log và có thể cần chiến lược khác.

Chọn độ sâu phù hợp

Mới bắt đầu · Chọn nhánh bằng thẻ khoảng cách

Đặt trước mặt các thẻ 15, 20, 30 và “lỗi”. Nói điều kiện đúng/sai rồi hành động; coi “lỗi” là trường hợp riêng.

Giải thích vì sao sau R phải nhìn lại trước khi F.

Đã học cơ bản · Tách phản xạ và tìm đường

Kiểm tra quy tắc cục bộ đúng bằng các ca biên. Sau đó thử sân có góc, hành lang cụt và vật cản khác nhau để xem nhiệm vụ có hoàn thành không.

Tạo một nhật ký gồm bước, d, nhánh, hướng; chỉ ra dấu hiệu robot lặp.

Muốn tìm hiểu sâu · Đưa giả định chuyển động vào ngưỡng

Trong mô hình số không có quán tính thực. Trong simulator 3D hoặc robot thật, cần xét độ trễ, tốc độ, khoảng dừng và vùng nhìn cảm biến. Bám tường hay BFS là chiến lược riêng, có dữ liệu và điều kiện áp dụng khác.

Mô tả thêm thông tin hoặc bộ nhớ cần có để tránh lặp ở ngõ cụt; phân biệt thông tin đó với chỉ tăng ngưỡng T.

Kiến thức trọng tâm

Điều kiện cục bộ

Một quy tắc đơn giản là: nếu d < T thì dừng rồi rẽ phải; nếu không thì tiến chậm. d và T phải cùng đơn vị. Với dấu <, d = T thuộc nhánh tiến; thay bằng ≤ là một thiết kế khác cần kiểm thử lại. Đọc không hợp lệ phải có nhánh dừng/báo lỗi, không được suy là đường trống.

Tránh va chạm không đồng nghĩa tìm được đích

Quy tắc chỉ nhìn phía trước và không lưu bản đồ có thể quay vòng ở ngõ cụt hoặc đi xa đích. Một lần rẽ phải chưa bảo đảm phía mới đã trống, vì vậy phải đọc lại trước khi tiến. Bám tường, BFS và lập bản đồ là các chiến lược khác, có điều kiện áp dụng riêng; không gọi quy tắc một cảm biến là bộ giải mọi mê cung.

Chạy có giới hạn và có thể giải thích

Đặt thời gian hoặc số bước tối đa, tiêu chí tới đích và nút dừng. Log khoảng cách, nhánh được chọn và trạng thái để biết vì sao robot lặp. Trong mô hình số, không có độ trễ/va chạm thật; trong simulator 3D hoặc phần cứng, ngưỡng còn cần dự phòng quán tính, tầm nhìn cảm biến và khoảng dừng.

Ví dụ có lời giải

T = 20 cm: d = 19 cm chọn dừng–rẽ; d = 20 và 21 cm chọn tiến theo dấu <. Nếu robot ở hành lang cụt, rẽ 90° có thể hướng vào tường bên; phải đo lại. Chương trình hết giới hạn mà chưa đạt đích phải báo chưa hoàn thành.

Lỗi thường gặp và cách tránh

1. Một khung simulator chạy được không chứng minh thuật toán tới đích hoặc tránh va chạm trong mọi sân.

2. Rẽ xong không có nghĩa phía mới trống; phải kiểm tra lại điều kiện trước khi tiến.

Các bước thực hành

1. Viết bảng dự đoán 19/20/21 cm với T = 20 cm; thêm một hàng dữ liệu không hợp lệ.

2. Dùng bàn cảm biến kiểm tra phép so sánh. Không coi kết quả nhánh là bằng chứng robot đã tránh được va chạm.

3. Trong simulator có cảm biến, chọn đúng đầu đo, triển khai dừng trước rẽ, đọc lại sau rẽ và chạy ở tốc độ thấp với giới hạn thời gian.

4. Tạo hoặc vẽ một ngõ cụt/vòng kín làm thuật toán không đến đích; ghi trạng thái lặp và điều kiện kết thúc thay vì tiếp tục chạy vô hạn.

Mã mẫu · Python · GearsBot (ev3dev2)

```
from math import pi, sqrt
from time import time, sleep
from ev3dev2.motor import MoveTank, OUTPUT_A, OUTPUT_B
from ev3dev2.sensor import INPUT_2, INPUT_4
from ev3dev2.sensor.lego import UltrasonicSensor
from ev3dev2.sensor.virtual import GPSSensor

tank = MoveTank(OUTPUT_A, OUTPUT_B)
gps = GPSSensor(INPUT_4)
WHEEL_DIAMETER_MM = 56.0
TRACK_WIDTH_MM = 152.0
SPEED = 25

def move_wheels(left_speed, right_speed, degrees, timeout_s=8.0):
  start_left = tank.left_motor.position
  start_right = tank.right_motor.position
  deadline = time() + timeout_s
  tank.on_for_degrees(left_speed, right_speed, degrees,
              brake=True, block=False)
  try:
    sleep(0.05)
    while time() < deadline:
      if tank.left_motor.is_stalled or tank.right_motor.is_stalled:
        raise RuntimeError("Motor stalled; check the course")
      if not (tank.left_motor.is_running or tank.right_motor.is_running):
        left_done = abs(tank.left_motor.position - start_left)
        right_done = abs(tank.right_motor.position - start_right)
        if min(left_done, right_done) >= degrees - 5:
          return
      sleep(0.02)
    raise RuntimeError("Motion timeout; reset before retrying")
  finally:
    tank.off(brake=True)

def forward(distance_mm):
  degrees = 360.0 * distance_mm / (pi * WHEEL_DIAMETER_MM)
  move_wheels(SPEED, SPEED, degrees)

def turn_right(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(SPEED, -SPEED, degrees)

def turn_left(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(-SPEED, SPEED, degrees)

distance_sensor = UltrasonicSensor(INPUT_2)
THRESHOLD_CM = 20.0
TIME_LIMIT_S = 20.0
MAX_ACTIONS = 40
# Goal is 60 cm straight ahead of the initial GPS position on the default pose.
start = gps.position
goal_x, goal_y = start[0], start[1] + 60
reason = "action_limit"
actions = 0

def read_distance_cm():
```

```
    value = distance_sensor.distance_centimeters
    if not 0 < value <= 255:
      raise ValueError("Invalid ultrasonic reading")
    return value

try:
    deadline = time() + TIME_LIMIT_S
    while actions < MAX_ACTIONS:
      if time() >= deadline:
        reason = "time_limit"
        break
      position = gps.position
      if sqrt((position[0] - goal_x) ** 2
            + (position[1] - goal_y) ** 2) <= 5:
        reason = "goal_reached"
        break
      distance_cm = read_distance_cm()
      if distance_cm < THRESHOLD_CM:
        turn_right()
      else:
        forward(50)
      actions += 1
      print("Action:", actions, "distance (cm):", round(distance_cm, 1))
    print("Stop reason:", reason)
    print("Goal (cm):", goal_x, goal_y, "GPS (cm):", gps.position[:2])
finally:
    tank.off(brake=True)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn robot Single Sensor Line Follower (singleFollower), mở Python và dán toàn bộ chương trình. Robot dùng bánh trái A, bánh phải B; đường kính bánh 56 mm, khoảng cách hai bánh 152 mm. Chọn sân rộng, phẳng; Reset trước mỗi lượt rồi Run. Dùng tư thế Reset mặc định hướng theo trục GPS y dương; đích đặt cách điểm đầu 60 cm theo trục này, bán kính chấp nhận 5 cm. Cảm biến khoảng cách ở INPUT_2. Đặt hộp chắn đường trên sân rồi thử: mỗi lần gặp vật dưới 20 cm robot rẽ phải, sau đó đọc lại cảm biến trước hành động tiếp theo. Mỗi bước tiến 50 mm; giới hạn 40 hành động và ngân sách 20 giây, một hành động đang chạy có timeout riêng 8 giây. Console phân biệt tới đích, hết thời gian và hết hành động; thử hành lang chữ U để quan sát giới hạn của quy tắc.

Bài tập áp dụng

Vẽ một hành lang chữ U có điểm đầu trong lòng chữ U và một đích nằm bên ngoài. Giải thích vì sao chỉ "gặp tường thì rẽ phải" chưa bảo đảm tới đích. Nêu hai thông tin/biện pháp bổ sung để nhận biết hoặc thoát hành vi lặp.

Gợi ý đối chiếu sau khi tự làm

Robot có thể lặp lại một nhóm vị trí/hướng mà không biết đang lặp vì chỉ dùng khoảng cách trước mặt. Có thể lưu trạng thái đã thăm hoặc bản đồ, bổ sung chiến lược khám phá có kiểm chứng và giới hạn số bước. Các bổ sung cần được thử trên bản đồ cụ thể; không tự bảo đảm mọi mê cung chỉ nhờ thêm một biến.

Tự kiểm tra mức hiểu

1. Dự đoán đúng nhánh ở d = T với dấu <.

2. Nêu được một tình huống phản ứng cục bộ bị lặp và một tiêu chí dừng.

3. Phân biệt được kết quả “không đâm tường trong lượt thử” với “tìm được đường tới đích”.

Câu hỏi trắc nghiệm

Quy tắc gặp vật cản thì rẽ phải có đảm bảo giải mọi mê cung?

1. Có

2. Không

3. Chỉ cần tăng tốc

Đáp án: 2. Quy tắc phản ứng cục bộ không xét toàn bộ bản đồ hoặc tiến độ tới đích, nên có thể bị vòng lặp hoặc bế tắc. Tăng tốc không bổ sung thông tin còn thiếu.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEXcode VR — môi trường lập trình chính thức

https://vr.vex.com/

VEXcode VR — tài liệu Python chính thức

https://api.vex.com/vr/home/python/

VEX CS Level 1 — bài học VEXcode VR

https://education.vex.com/stemlabs/cs/cs-level-1-vexcode-vr

GEARS — cấu hình Single Sensor Line Follower

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GEARS — API ev3dev2.motor trong simulator

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GEARS — API GPS và đơn vị tọa độ

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/virtual.py

GEARS — API cảm biến siêu âm

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/lego.py

Bài 4/6 · Thử thách: giao hàng ảo

Thiết kế hai tuyến giao hàng cùng đích và xác định chi phí trước khi thử.

Chuẩn bị

Hoàn thành chuỗi lệnh, vòng lặp và tránh vật cản. Biết tính tổng chi phí có trọng số và phân biệt thời gian mô hình với thời gian đo.

Sau bài học, bạn có thể

1. Thiết kế hai tuyến giao hàng cùng đích và xác định chi phí trước khi thử.

2. So sánh số bước, số lần rẽ và thời gian theo cùng quy ước.

3. Chọn phương án đáp ứng nhiệm vụ thay vì chỉ tối ưu một chỉ số đẹp.

Cùng hiểu bài từ đầu

Hai tuyến tới cùng đích có thể khác nhau về số ô, số rẽ và thời gian. Trước khi chọn tuyến "tốt hơn", hãy chọn điều cần tối ưu và quy ước tính. Bài này cho thấy một tuyến đi xa hơn vẫn có thể nhanh hơn khi rẽ tốn nhiều thời gian.

Hai con đường tới trường: đường ngắn phải chờ nhiều lần ở giao lộ, đường vòng ít chờ hơn. Chỉ so chiều dài chưa đủ để biết đường nào ít thời gian hơn.

Trong mô hình robot, mỗi F và mỗi R có thể có chi phí khác nhau. Tổng chi phí được tính từ số lần từng loại hành động, sau khi đã kiểm tra tuyến tới đúng đích.

Giới hạn của ví von: Giao thông đời thực phức tạp hơn hai hằng số. Mô hình chi phí cố định chỉ là ước lượng; thời gian đo thật còn phụ thuộc tăng tốc, cảm biến và cách simulator thực hiện lệnh.

Những từ cần hiểu

1. Chi phí hành động: Giá trị gán cho từng loại hành động, ví dụ một F tốn 1 s và một R tốn 3 s.

2. Tuyến hợp lệ: Đạt đích và đáp ứng ràng buộc sân, vùng cấm, hướng cuối nếu có, cùng giới hạn đã đặt.

3. Số bước và số khối: Một khối có thể yêu cầu đi nhiều ô. Đếm khối và đếm quãng đường là hai quy ước khác nhau.

4. So sánh công bằng: Giữ cùng đề bài, trạng thái đầu và quy tắc đo khi so các phương án.

Cùng đích, ít bước tiến hơn vẫn có thể tốn thời gian hơn

Bắt đầu với: Lưới 5×4, không vật cản, bắt đầu (0,0,đông), đích (3,2), không yêu cầu hướng cuối. Tuyến A: F,R,F,L,F,F,R,F. Tuyến B: F,F,F,F,R,F,F,R,F. Giả sử F tốn 1 s, mỗi lần rẽ tốn 3 s.

Kiểm tra cả hai tuyến tới đích

A đi lần lượt qua (1,0), (1,1), (3,1) rồi (3,2). B đi qua (4,0), (4,2) rồi quay về (3,2). Cả hai ở trong lưới và tới đích; hướng cuối lần lượt nam và tây đều được phép.

Đếm riêng tiến và rẽ

A có 5 F và 3 lần rẽ. B có 7 F và 2 lần rẽ. B đi nhiều hơn 2 ô nhưng ít hơn 1 lần rẽ.

Áp dụng cùng mô hình chi phí

A: 5 × 1 + 3 × 3 = 14 s. B: 7 × 1 + 2 × 3 = 13 s. Trong mô hình này B có thời gian thấp hơn 1 s.

Kết quả: A ngắn hơn theo số ô; B nhanh hơn theo mô hình thời gian đã chọn. Kết luận phải đi kèm chỉ số và giả định.

Nếu đổi chi phí rẽ, thứ tự tốt hơn có thể đổi. Bạn đang chọn lời giải cho một mục tiêu xác định, không tìm một tuyến luôn tốt nhất theo mọi tiêu chí.

Tự giải thích rồi đối chiếu

Nếu một lần rẽ chỉ tốn 1 s, tuyến nào có thời gian mô hình thấp hơn?

A tốn 5 + 3 = 8 s; B tốn 7 + 2 = 9 s, nên A thấp hơn. Chi phí đã thay đổi nên kết luận thay đổi.

Một tuyến ngắn hơn nhưng vượt biên có được so như phương án thành công không?

Không. Trước hết tuyến phải hợp lệ. Có thể ghi lượt đó là thất bại, nhưng không dùng thời gian ngắn của nó làm bằng chứng hoàn thành tốt hơn.

Chọn độ sâu phù hợp

Mới bắt đầu · Dùng hai cột F và rẽ

Mỗi lần gặp F thêm một dấu vào cột tiến, gặp R/L thêm vào cột rẽ. Nhân từng cột với giá của nó rồi cộng.

Tự xác nhận 5 F + 3 rẽ của A bằng cách đi từng lệnh trên giấy.

Đã học cơ bản · Tìm điều kiện đổi thứ tự

Với cùng hai tuyến, A có thêm 1 rẽ nhưng bớt 2 F. So lợi ích bớt tiến với chi phí thêm rẽ thay vì chỉ thử một bảng giá.

Tìm giá trị tR làm hai tuyến hòa nhau nếu tF = 1 s.

Muốn tìm hiểu sâu · Định nghĩa đúng mục tiêu tối ưu

Nếu hướng cuối cũng bị ràng buộc, hai tuyến trên có thể không còn tương đương. Tối ưu thời gian cần trạng thái và chi phí đủ mô tả cả quay lẫn tiến; tối ưu số khối còn khác nữa. Dùng thời gian đo để kiểm tra mức phù hợp của mô hình cố định.

Đề xuất cách báo đồng thời tỷ lệ hoàn thành, số ô, số rẽ và thời gian mà không tráo chỉ số khi chọn tuyến.

Kiến thức trọng tâm

Nhiệm vụ và ràng buộc

Một bài giao hàng cần điểm đầu/hướng đầu, đích, vật cản, vùng không đi qua và điều kiện dừng. Hai giải pháp chỉ so sánh được khi dùng cùng đề bài. Trước hết phải kiểm tra đến đúng đích và không vi phạm ràng buộc; chương trình ngắn nhưng đi sai không được coi là thắng.

Chi phí là một mô hình được chọn

Nếu một bước tiến tốn tF giây và một lần rẽ tốn tR giây, ước lượng T = nF × tF + nR × tR. nF và nR là số bước tiến và rẽ. Đây là mô hình rời rạc; thời gian thật còn có tăng/giảm tốc, xử lý cảm biến và cách nền tảng chạy lệnh. Đường ít bước tiến có thể tốn nhiều thời gian hơn vì rẽ nhiều.

Đo trước và sau cải tiến

Đặt lại cùng trạng thái đầu cho từng lượt, lưu phiên bản chương trình và ghi cả thất bại. Đếm theo cùng quy ước: một lệnh tiến 3 ô có thể là một khối lập trình nhưng ba ô quãng đường; không tráo "số khối" và "số hành động" giữa A/B. Chọn chỉ số phù hợp mục tiêu của đề bài.

Ví dụ có lời giải

Cùng xuất phát (0,0,đông), đích (3,2), hướng cuối được phép khác nhau. Tuyến A: F,R,F,L,F,F,R,F có 5 bước tiến, 3 lần rẽ; tuyến B: F,F,F,F,R,F,F,R,F có 7 bước tiến, 2 lần rẽ. Cả hai đều ở trong lưới 5×4 và đến đúng đích. Nếu tF = 1 s, tR = 3 s thì TA = 14 s, TB = 13 s; B đi nhiều ô hơn nhưng có chi phí thời gian mô hình thấp hơn.

Lỗi thường gặp và cách tránh

1. Tuyến ít ô hơn không luôn nhanh hơn nếu rẽ có chi phí đáng kể.

2. Chương trình có ít khối hơn không nhất thiết có ít hành động hoặc quãng đường hơn.

Các bước thực hành

1. Viết đặc tả điểm đầu/hướng đầu/đích và chốt chỉ số: thành công, số bước, số rẽ, chi phí thời gian.

2. Tạo tuyến A trong bàn lưới và dự đoán các trạng thái quan trọng. Reset, chạy và ghi đầy đủ kết quả.

3. Tạo tuyến B với cùng đích/ràng buộc; dùng cùng quy ước đếm rồi tính T theo tF,tR đã chốt.

4. Chuyển một tuyến sang simulator nếu muốn đo thời gian thực của môi trường đó. Báo riêng thời gian tính theo mô hình và thời gian đo, không coi hai số là một.

Mã mẫu · Python 3 · kiểm tra tuyến và chi phí

```
# Both routes start at (0, 0, east). The goal checks position only.
WIDTH, HEIGHT = 5, 4
START = (0, 0, 0)
GOAL = (3, 2)
DIRECTIONS = [(1, 0), (0, 1), (-1, 0), (0, -1)]
ROUTES = {"A": "F R F L F F R F", "B": "F F F F R F F R F"}

def evaluate(commands, forward_s, turn_s):
  x, y, heading = START
  moves = turns = 0
  for command in commands.split():
    if command == "F":
      dx, dy = DIRECTIONS[heading]
      x, y = x + dx, y + dy
      moves += 1
      if not (0 <= x < WIDTH and 0 <= y < HEIGHT):
        return False, moves, turns, None
    elif command in ("L", "R"):
      heading = (heading + (1 if command == "R" else -1)) % 4
      turns += 1
    else:
      raise ValueError("Unknown command")
  success = (x, y) == GOAL
  return success, moves, turns, moves * forward_s + turns * turn_s

for forward_s, turn_s in [(1.0, 3.0), (2.0, 0.2)]:
  print("Model seconds per forward/turn:", forward_s, turn_s)
  candidates = []
  for name, route in ROUTES.items():
    success, moves, turns, cost = evaluate(route, forward_s, turn_s)
    print(name, "success:", success, "forward:", moves,
      "turns:", turns, "model seconds:", cost)
    if success:
      candidates.append((cost, name))
  print("Lowest-cost successful route:", min(candidates)[1] if candidates else None)
```

Chọn và sử dụng phòng lab

Chạy bằng editor Python của GearsBot hoặc Python 3; code mô phỏng lưới và xuất bảng chi phí, không điều khiển motor. Hai tuyến trong code đều được kiểm tra tới (3,2), hướng cuối được phép khác nhau. A có 5 tiến, 3 rẽ; B có 7 tiến, 2 rẽ. Thay giá tiến/rẽ để thấy lựa chọn đổi từ B sang A. Giá thời gian là giả thiết của mô hình; khi chạy robot trên sân cần đo thời gian thực và kiểm tra đích riêng.

Bài tập áp dụng

Với hai tuyến trong ví dụ, A có 5 tiến, 3 rẽ và B có 7 tiến, 2 rẽ. Tính chi phí khi tF = 2 s, tR = 0,2 s; phương án được chọn có đổi không? Nếu yêu cầu cả hai cùng kết thúc hướng nam, cần thêm lệnh gì cho B và chi phí thay đổi ra sao? Nếu B không tới đích thì xử lý so sánh thế nào?

Gợi ý đối chiếu sau khi tự làm

TA = 5×2 + 3×0,2 = 10,6 s; TB = 7×2 + 2×0,2 = 14,4 s nên A được chọn, khác ví dụ trước. B kết thúc hướng tây; thêm L để quay về nam, chi phí mới là 14,6 s. Nếu B không tới đích thì ghi thất bại và loại khỏi lựa chọn đạt nhiệm vụ; vẫn giữ kết quả đó trong báo cáo.

Tự kiểm tra mức hiểu

1. Đặc tả có cùng điểm đầu/hướng đầu/đích cho A và B.

2. Tính đúng chi phí và ghi rõ đơn vị giây cùng giả định.

3. Giải thích được vì sao đổi trọng số chi phí có thể đổi phương án tốt hơn.

Câu hỏi trắc nghiệm

Khi so sánh số lệnh cần...

1. Bỏ qua lệnh rẽ

2. Đếm theo một quy ước nhất quán

3. Chọn phương án đẹp hơn

Đáp án: 2. Một quy ước đếm nhất quán giúp các giá trị so sánh cùng một đại lượng. Bỏ rẽ ở một phương án nhưng tính ở phương án kia sẽ tạo kết luận sai.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEXcode VR — môi trường lập trình chính thức

https://vr.vex.com/

VEXcode VR — tài liệu Python chính thức

https://api.vex.com/vr/home/python/

VEX CS Level 1 — bài học VEXcode VR

https://education.vex.com/stemlabs/cs/cs-level-1-vexcode-vr

Python 3 — hàm có sẵn và kiểu dữ liệu

https://docs.python.org/3/library/functions.html

Bài 5/6 · Tìm đường trên lưới bằng BFS

Mô tả BFS như phép duyệt theo lớp sử dụng hàng đợi FIFO và tập đã thăm.

Chuẩn bị

Biết tọa độ lưới, ô vật cản, hàng đợi vào trước–ra trước và cách biểu diễn một tuyến bằng danh sách ô.

Sau bài học, bạn có thể

1. Mô tả BFS như phép duyệt theo lớp sử dụng hàng đợi FIFO và tập đã thăm.

2. Dựng lại một đường đi bằng thông tin cha và xử lý trường hợp không có đường.

3. Nêu đúng điều kiện tối ưu của BFS và giới hạn khi chi phí rẽ khác chi phí tiến.

Cùng hiểu bài từ đầu

BFS tìm đường bằng cách mở rộng dần: mọi ô cách điểm đầu một bước, rồi hai bước, rồi ba bước. Hàng đợi giữ thứ tự đó. Để hiểu thuật toán, bạn sẽ theo dõi một hàng đợi rất nhỏ và xem "lưu cha" giúp dựng lại đường đi ra sao.

Một nhóm người tìm đồ thất lạc bằng cách kiểm tra tất cả khu vực sát điểm bắt đầu, rồi các khu xa hơn từng lớp. Các khu đã xếp lịch kiểm tra được đánh dấu để không xếp lại.

BFS cũng đánh dấu ngay khi đưa ô vào hàng đợi. Hàng đợi FIFO đưa ô được thêm sớm ra xét sớm, nhờ đó các lớp theo số bước không bị đảo thứ tự.

Giới hạn của ví von: BFS của bài nhận bản đồ đã biết. Nó không tự nhìn ra tường bằng cảm biến và không tự tối ưu thời gian quay; điều bảo đảm ngắn nhất chỉ đúng khi các cạnh có cùng chi phí.

Những từ cần hiểu

1. Đồ thị lưới: Mỗi ô đi được là một đỉnh; các ô kề theo bốn hướng có cạnh nối nếu cho phép đi qua.

2. Hàng đợi FIFO: Thêm ở cuối, lấy ở đầu; phần tử vào trước được lấy ra trước.

3. Tập đã thăm: Ghi các ô đã được đưa vào hàng đợi để tránh lặp lại, không đợi đến lúc lấy ra mới đánh dấu.

4. Cha · parent: Với mỗi ô mới, lưu ô đã dẫn tới nó. Lần ngược các cha từ đích rồi đảo thứ tự sẽ cho đường từ đầu tới đích.

Theo hàng đợi trên lưới 3×2 có một ô cấm

Bắt đầu với: Các ô có x = 0..2, y = 0..1. Bắt đầu S = (0,0), đích G = (2,0), ô cấm (1,0). Xét láng giềng theo đông, nam, tây, bắc; mọi bước có chi phí bằng 1.

Khởi tạo và xét ô đầu

Hàng đợi ban đầu [(0,0)], đã thăm có (0,0). Lấy (0,0): phía đông bị cấm, phía nam (0,1) hợp lệ. Đưa (0,1) vào hàng đợi và đặt cha của nó là (0,0).

Mở rộng từng ô theo FIFO

Lấy (0,1), thêm (1,1), lưu cha (0,1). Lấy (1,1), thêm (2,1), lưu cha (1,1). Những ô đã thăm hoặc ngoài lưới không được thêm lại.

Gặp đích và lần ngược cha

Lấy (2,1), phát hiện (2,0) và lưu cha (2,1). Từ đích lần ngược: (2,0) ← (2,1) ← (1,1) ← (0,1) ← (0,0). Đảo lại để có đường đi.

Kết quả: Đường tìm được là (0,0) → (0,1) → (1,1) → (2,1) → (2,0), gồm 4 bước. Khoảng cách Manhattan trực tiếp là 2 nhưng ô cấm buộc đi vòng.

BFS tìm đường ít cạnh nhất trong bản đồ và mô hình chi phí bằng nhau này. Đổi thứ tự láng giềng có thể chọn đường khác cùng độ dài, nhưng không biến BFS thành tối ưu mọi loại chi phí.

Tự giải thích rồi đối chiếu

Nếu hàng đợi rỗng mà chưa gặp đích thì kết luận gì?

Không có đường trong phần bản đồ và luật di chuyển đã mô hình hóa. Chương trình phải báo kết quả đó thay vì dựng một đường giả.

Tại sao đánh dấu ngay khi thêm vào hàng đợi?

Một ô có thể được nhiều ô kề phát hiện. Đánh dấu ngay ngăn thêm trùng và giữ một quan hệ cha nhất quán cho lần phát hiện đầu.

Chọn độ sâu phù hợp

Mới bắt đầu · Diễn thuật toán bằng thẻ ô

Viết các tọa độ lên thẻ. Thêm thẻ mới vào cuối hàng, luôn lấy thẻ đầu; vẽ mũi tên cha ngay khi thêm. Chưa cần thuộc mã Python.

Dùng ba bước của ví dụ để tự vẽ lại đường từ đích ngược về đầu.

Đã học cơ bản · Phân biệt bản đồ, đường và lệnh

Đường BFS là danh sách ô. Muốn robot chạy cần đổi mỗi cạnh thành hướng di chuyển rồi thành lệnh tiến/rẽ theo hướng hiện tại. Số ô trên đường bằng số bước cộng một nếu đường có cả đầu và đích.

Đếm số cạnh của ví dụ và viết chuỗi F/R/L hợp lệ khi robot bắt đầu hướng đông.

Muốn tìm hiểu sâu · Kiểm tra mô hình tối ưu và chi phí tìm kiếm

BFS trên (x,y) tối ưu số cạnh đồng giá. Khi rẽ có giá, cần biểu diễn thêm hướng và dùng thuật toán theo trọng số phù hợp. Số ô mở rộng là công sức tìm kiếm, còn độ dài đường là chi phí lời giải; hai số không đồng nghĩa. Nếu áp giới hạn mở rộng, hết ngân sách là chưa tìm xong, không tự chứng minh không có đường.

Nêu một ca không có đường và một ca có đường nhưng ngân sách mở rộng quá nhỏ; xác định thông báo khác nhau cần có.

Kiến thức trọng tâm

Từ sân thành đồ thị

Mỗi ô trống là một đỉnh; nối cạnh giữa hai ô kề theo bốn hướng nếu robot được phép đi giữa chúng. Ô vật cản không được thêm vào láng giềng. Bản đồ này là dữ liệu đầu vào; BFS không tự khám phá tường hoặc đọc cảm biến. Nếu robot chưa biết sân, cần một bước thu thập/xây dựng bản đồ riêng.

Duyệt theo lớp và lưu cha

Bắt đầu với hàng đợi chứa ô đầu và đánh dấu đã thăm. Lấy phần tử đầu hàng đợi, xét từng láng giềng hợp lệ chưa thăm, đánh dấu ngay khi đưa vào hàng đợi và lưu parent. Các ô cách một bước được xét trước các ô cách hai bước. Khi gặp đích, lần theo parent ngược về điểm đầu rồi đảo danh sách; nếu hàng đợi rỗng mà chưa tới đích, phải báo không có đường.

Tối ưu đúng đại lượng

BFS tối ưu số cạnh khi mọi cạnh có cùng chi phí. Trên lưới bốn hướng không vật cản, khoảng cách Manhattan $|\Delta x| + |\Delta y|$ cho số bước tối thiểu; vật cản có thể làm đường dài hơn hoặc không tồn tại. Nếu hướng/rẽ ảnh hưởng chi phí, mô hình trạng thái chỉ (x,y) chưa đủ để tối ưu thời gian; có thể cần (x,y,hướng) và thuật toán theo trọng số phù hợp.

Ví dụ có lời giải

Từ (0,0) đến (3,2) trên lưới không vật cản cần ít nhất $|3|+|2| = 5$ bước. Hai tuyến E,E,E,S,S và S,S,E,E,E đều có 5 bước; số lần rẽ khi khởi đầu hướng đông có thể khác. BFS không tự chọn tuyến có thời gian nhỏ nhất nếu giá rẽ chưa được đưa vào mô hình.

Lỗi thường gặp và cách tránh

1. BFS không tự cung cấp bản đồ và không phải quy tắc “gặp tường thì rẽ”.

2. BFS tối ưu số bước đồng chi phí, không tự tối ưu thời gian có phạt rẽ hoặc năng lượng robot thật.

Các bước thực hành

1. Vẽ lưới 5×4, chọn điểm đầu/đích và tô ít nhất hai ô vật cản. Quy định chỉ đi bốn hướng, không đi chéo.

2. Duyệt trên giấy theo lớp; ghi nội dung hàng đợi và parent. Đánh dấu ô ngay khi thêm hàng đợi để tránh đưa lặp.

3. Chạy hàm Python thuần, đối chiếu đường trả về. Chặn kín đích để xác nhận hàm trả kết quả không có đường.

4. Chuyển danh sách ô thành tiến/rẽ theo hướng đầu và thử trên bàn lưới hoặc simulator phù hợp; kiểm tra biên và tỷ lệ đơn vị khi chuyển môi trường.

Mã mẫu · Python 3 · tìm đường BFS

```
WIDTH, HEIGHT = 5, 4
BLOCKED = {(1, 1), (2, 1)}
START, GOAL = (0, 0), (3, 2)

def bfs(start, goal, blocked, width, height):
  def valid(cell):
    x, y = cell
    return 0 <= x < width and 0 <= y < height and cell not in blocked

  if not valid(start) or not valid(goal):
    return None
  queue = [start]
  head = 0
  parent = {start: None}
  while head < len(queue):
    current = queue[head]
    head += 1
    if current == goal:
      path = []
      while current is not None:
        path.append(current)
        current = parent[current]
      return path[::-1]
    x, y = current
    for neighbor in [(x + 1, y), (x, y + 1), (x - 1, y), (x, y - 1)]:
      if valid(neighbor) and neighbor not in parent:
        parent[neighbor] = current
        queue.append(neighbor)
  return None

path = bfs(START, GOAL, BLOCKED, WIDTH, HEIGHT)
print("Path:", path)
print("Forward steps:", len(path) - 1 if path is not None else None)
```

Chọn và sử dụng phòng lab

Chạy bằng editor Python của GearsBot hoặc Python 3; console in danh sách tọa độ từ đầu tới đích và số bước. Code dùng hàng đợi FIFO bằng list và chỉ số head. Để làm bài tập, đổi WIDTH, HEIGHT thành 4, 3 và BLOCKED thành {(1,0),(1,1)}; chạy lại rồi thêm (1,2) để kiểm tra trường hợp bị chặn. None nghĩa là bản đồ đã khai báo không có đường hợp lệ. Muốn robot di chuyển theo đường tìm được, cần quy đổi ô sang mm và tọa độ sang hướng của sân.

Bài tập áp dụng

Dùng lưới 4×3, đầu (0,0), đích (3,2), vật cản {(1,0),(1,1)}. Tìm một đường ngắn nhất và số bước; sau đó thêm vật cản (1,2) để chặn toàn cột x = 1. Nêu kết quả mà chương trình cần trả về.

Gợi ý đối chiếu sau khi tự làm

Một đường ngắn nhất là (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (3,2), gồm 5 bước. Khi chặn thêm (1,2), không có đường sang phía đích trong lưới bốn hướng; hàm trả None, không trả danh sách trống như thể đã tới đích. Đổi đường thành lệnh cần xét hướng đầu.

Tự kiểm tra mức hiểu

1. Chỉ ra được FIFO, tập đã thăm và parent trong mã.

2. Chạy được ca start = goal và ca không có đường mà không nhầm là thành công.

3. Nêu đúng giả định cạnh đồng chi phí và giải thích vì sao cần xét hướng nếu muốn tính giá rẽ.

Câu hỏi trắc nghiệm

BFS tối ưu số bước khi...

1. Mọi cạnh cùng chi phí

2. Mọi tốc độ khác nhau

3. Không cần bản đồ

Đáp án: 1. FIFO duyệt các lớp theo số cạnh tăng dần, nên đường đầu tiên tới đích ngắn nhất theo số cạnh khi mỗi cạnh cùng chi phí. Nếu chi phí khác nhau, thứ tự lớp không còn tương đương chi phí nhỏ nhất.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEXcode VR — môi trường lập trình chính thức

https://vr.vex.com/

VEXcode VR — tài liệu Python chính thức

https://api.vex.com/vr/home/python/

VEX CS Level 1 — bài học VEXcode VR

https://education.vex.com/stemlabs/cs/cs-level-1-vexcode-vr

Python 3 — hàm có sẵn và kiểu dữ liệu

https://docs.python.org/3/library/functions.html

Bài 6/6 · Thực nghiệm: so sánh hai thuật toán

Thiết kế phép so sánh công bằng hai thuật toán và chốt tiêu chí trước khi chạy.

Chuẩn bị

Hoàn thành thử thách chi phí và tìm đường BFS; biết tỷ lệ phần trăm, bảng dữ liệu và khái niệm đầu vào được giữ cố định.

Sau bài học, bạn có thể

1. Thiết kế phép so sánh công bằng hai thuật toán và chốt tiêu chí trước khi chạy.

2. Báo cáo tỷ lệ hoàn thành cùng chi phí và các lượt thất bại, không chọn riêng lượt đẹp.

3. Phân biệt kiểm thử trên sân đã điều chỉnh với kiểm thử khả năng tổng quát trên sân mới.

Cùng hiểu bài từ đầu

Muốn biết một thuật toán tốt hơn thuật toán khác, bạn cần một phép thử có thể giải thích và lặp lại. Chọn sân, dữ liệu mà mỗi thuật toán được biết và tiêu chí trước khi chạy. Đừng chỉ lưu những lần thành công hoặc những chỉ số có lợi cho phương án mình thích.

Hai bạn làm cùng một đề nhưng một bạn được xem trước đáp án thì điểm số không chỉ phản ánh cách giải. Muốn so cách làm, cần nói rõ điều kiện và thông tin mà mỗi người nhận được.

Thuật toán robot cũng vậy: cùng bản đồ, cùng trạng thái đầu và cùng ngân sách mới tạo nền so sánh rõ ràng. Nếu một thuật toán có bản đồ đầy đủ còn thuật toán kia chỉ có cảm biến, phải báo khác biệt ấy.

Giới hạn của ví von: Thử nghiệm vẫn chỉ hỗ trợ kết luận trong phạm vi bộ sân và điều kiện đã xét. Lặp cùng một mô phỏng xác định nhiều lần không tạo thêm nhiều tình huống độc lập.

Những từ cần hiểu

1. Tỷ lệ hoàn thành: Số sân hoàn thành đúng chia tổng số sân thử, nhân 100%. Timeout và sai đích vẫn có trong mẫu số.

2. Ngân sách tìm kiếm: Giới hạn tài nguyên như số ô mở rộng. Cùng giới hạn giúp so công sức xử lý, nhưng cần ghi ca hết ngân sách riêng.

3. Biến được giữ cố định: Điều kiện chung như bản đồ, thứ tự láng giềng, vị trí đầu, đích và cách tính chi phí.

4. Bộ kiểm thử chưa dùng để chỉnh: Các sân được giữ lại đến đánh giá cuối, giúp xem cải tiến có dùng được ngoài các sân đã quen hay không.

Tách hoàn thành, độ dài đường và công sức tìm kiếm

Bắt đầu với: Trên ba sân chung, A thành công cả 3, đường dài 6, 8, 10 bước. B thành công hai sân đầu với đường 6, 9 bước; sân thứ ba hết ngân sách mở rộng. Cả hai nhận cùng bản đồ đầy đủ và cùng giới hạn.

Ghi tỷ lệ hoàn thành trước

A đạt 3/3 = 100%; B đạt 2/3 ≈ 66,7%. Không bỏ sân B hết ngân sách ra khỏi mẫu số.

So đường trên các sân chung đều thành công

Ở hai sân đầu, A có đường 6 và 8 bước; B có 6 và 9. Trung bình trên hai sân này là 7 bước cho A và 7,5 cho B. Báo rõ đang dùng đúng hai sân chung, không trộn với cả ba của A.

Báo riêng công sức tìm kiếm

Nếu ghi số ô mở rộng, đặt cột riêng. Thuật toán có thể mở rộng ít ô mà tìm đường dài hơn; số ô mở rộng không phải độ dài đường. Hết ngân sách nghĩa chưa có lời giải trong giới hạn, không đủ kết luận sân vô nghiệm.

Kết quả: Báo cáo cần nêu A hoàn thành 3/3, B 2/3, kết quả đường trên hai sân chung và các số mở rộng nếu có. Mỗi chỉ số trả lời một câu hỏi riêng.

Kết luận có giới hạn rõ: đây là kết quả trên ba sân đã thử, không chứng minh A luôn thắng trên mọi sân. Tiếp theo cần thêm cấu trúc sân khác và bộ kiểm thử chưa dùng khi chỉnh thuật toán.

Tự giải thích rồi đối chiếu

Chạy lại cùng một sân xác định 20 lần có tương đương thử 20 sân khác nhau không?

Không. Các lần lặp có thể giúp phát hiện lỗi thao tác nhưng thường cho cùng kết quả đầu ra. Muốn đánh giá phạm vi hoạt động cần thêm tình huống khác nhau.

Vì sao phải chốt tiêu chí trước khi chạy?

Để tránh chỉ chọn chỉ số có lợi sau khi thấy kết quả. Cần định nghĩa trước thế nào là thành công, cách tính timeout và phạm vi tính trung bình.

Chọn độ sâu phù hợp

Mới bắt đầu · Dùng một bảng ngắn, ghi cả thất bại

Mỗi dòng là một sân; điền đạt/chưa đạt trước, rồi số bước và lý do lỗi. Bạn có thể bắt đầu bằng hai hoặc ba sân nhỏ thay vì bảng thống kê dài.

Giải thích vì sao 2 thành công trên 3 lượt phải là 2/3, không phải 2/2.

Đã học cơ bản · So thuật toán với cùng đầu vào

Khi so BFS và DFS trong bàn thí nghiệm, giữ cùng bản đồ đầy đủ, đầu/đích, thứ tự xét láng giềng và giới hạn mở rộng. Theo dõi cả đường tìm được lẫn số ô mở rộng.

Tạo một sân có nhánh cụt rồi dự đoán vì sao độ dài lời giải và số ô mở rộng có thể xếp hạng hai thuật toán khác nhau.

Muốn tìm hiểu sâu · Tách chỉnh thuật toán và đánh giá cuối

Dùng một nhóm sân để phát triển, giữ một nhóm khác để đánh giá. Với yếu tố ngẫu nhiên cần ghi seed hoặc quy tắc sinh cùng số lần thử. Báo cả phân bố chi phí và độ tin cậy, không chỉ một trung bình che thất bại.

Viết một tiêu chí chọn phương án gồm yêu cầu tối thiểu về hoàn thành, rồi cách so chi phí giữa các phương án đáp ứng yêu cầu ấy.

Kiến thức trọng tâm

So sánh cùng bài toán và cùng thông tin

Giữ cùng bản đồ, robot, vị trí/hướng đầu, giới hạn thời gian, tốc độ và quy ước đo khi so A/B. Nếu A chỉ thấy cảm biến còn B được cấp sẵn bản đồ đầy đủ, phải công bố sự khác nhau về thông tin; kết quả không chỉ phản ánh thuật toán. Chốt tiêu chí trước: tới đích đúng ràng buộc, số bước, số rẽ và thời gian.

Báo cáo đầy đủ

Tỷ lệ hoàn thành = số sân hoàn thành / tổng số sân thử × 100%. Ghi các lượt timeout, vượt biên hoặc sai đích như kết quả thất bại. Khi so thời gian, báo rõ đang tính trên mọi lượt với quy tắc timeout hay chỉ các lượt thành công; không trộn hai cách. Có thể ưu tiên độ tin cậy rồi mới xét chi phí trong số phương án đáp ứng yêu cầu.

Khả năng lặp lại và khả năng tổng quát

Mô phỏng xác định chạy lại cùng đầu vào thường cho cùng đầu ra; đó không phải nhiều tình huống độc lập. Dùng các sân có cấu trúc khác nhau, giữ một vài sân chưa dùng để chỉnh thuật toán rồi kiểm thử cuối. Với môi trường ngẫu nhiên phải ghi seed/quy tắc sinh sân và số lần thử; không suy rộng vượt quá dữ liệu đã có.

Ví dụ có lời giải

Trên 5 sân chung: A hoàn thành 4/5 = 80%, B hoàn thành 5/5 = 100%. A có thể nhanh hơn ở bốn sân đã thành công, nhưng chưa có căn cứ gọi A tốt hơn toàn diện vì A thất bại một sân. Báo cả tỷ lệ hoàn thành, chi phí và thông tin đầu vào của mỗi thuật toán.

Lỗi thường gặp và cách tránh

1. Chạy lại cùng sân xác định 100 lần không tự thay thế 100 sân khác nhau.

2. Trung bình thời gian chỉ trên lượt thành công không mô tả đầy đủ độ tin cậy khi có timeout.

Các bước thực hành

1. Chốt A/B, nguồn thông tin của mỗi thuật toán và các tiêu chí trước khi chạy. Định nghĩa rõ timeout, tới đích và vi phạm ràng buộc.

2. Chọn ít nhất ba sân có cấu trúc khác nhau: trống, có ngõ cụt, có nhiều tuyến. Giữ thêm một sân chưa dùng để chỉnh thuật toán.

3. Với mỗi sân, đặt lại cùng vị trí/hướng cho A/B, chạy có giới hạn và ghi mọi kết quả; không bỏ dòng thất bại.

4. Tính tỷ lệ hoàn thành, so chi phí theo cùng quy tắc, rồi chạy sân chưa dùng. Viết kết luận giới hạn ở bộ sân đã thử và nêu bước kiểm tra tiếp theo.

Mã mẫu · Python 3 · thực nghiệm BFS và DFS

```
# Deterministic grid experiments. Times below are declared action costs.
WIDTH, HEIGHT = 5, 4
START, GOAL = (0, 0), (4, 3)
MAPS = [set(), {(1, 0), (1, 1), (1, 2)},
    {(2, 0), (2, 1), (2, 2), (2, 3)}]
FORWARD_S, TURN_S = 1.0, 0.5
EXPANSION_LIMIT = 100
DIRECTIONS = [(1, 0), (0, 1), (-1, 0), (0, -1)]

def search(blocked, mode):
  frontier = [START]
  parent = {START: None}
  expanded = 0
  while frontier and expanded < EXPANSION_LIMIT:
    current = frontier.pop(0) if mode == "BFS" else frontier.pop()
    expanded += 1
    if current == GOAL:
      path = []
      while current is not None:
        path.append(current)
        current = parent[current]
      return path[::-1], expanded, "goal_reached"
    x, y = current
    for dx, dy in DIRECTIONS:
      neighbor = (x + dx, y + dy)
      nx, ny = neighbor
      if (0 <= nx < WIDTH and 0 <= ny < HEIGHT
          and neighbor not in blocked and neighbor not in parent):
        parent[neighbor] = current
        frontier.append(neighbor)
  reason = "expansion_limit" if frontier else "no_path"
  return None, expanded, reason

def path_cost(path):
  heading = 0  # east
  turns = 0
  for current, following in zip(path, path[1:]):
    delta = (following[0] - current[0], following[1] - current[1])
    next_heading = DIRECTIONS.index(delta)
    change = (next_heading - heading) % 4
    turns += min(change, 4 - change)
    heading = next_heading
  steps = len(path) - 1
  return steps, turns, steps * FORWARD_S + turns * TURN_S

for mode in ["BFS", "DFS"]:
  completed = 0
  successful_costs = []
  for map_id, blocked in enumerate(MAPS, 1):
    path, expanded, reason = search(blocked, mode)
    if path is None:
      print(mode, "map", map_id, reason, "expanded:", expanded)
    else:
      steps, turns, cost_s = path_cost(path)
      completed += 1
      successful_costs.append(cost_s)
      print(mode, "map", map_id, reason, "steps:", steps,
```

```
            "turns:", turns, "model seconds:", cost_s,
            "expanded:", expanded)
    print(mode, "completion:", completed, "/", len(MAPS))
    mean_cost = sum(successful_costs) / completed if completed else None
    print("Mean model seconds on successful maps:", mean_cost)
```

Chọn và sử dụng phòng lab

Chạy bằng editor Python của GearsBot hoặc Python 3. Code so sánh BFS và DFS trên cùng ba bản đồ, cùng thứ tự xét láng giềng và cùng thông tin toàn bản đồ. Console ghi kết quả từng sân, số ô đã xét, số bước/rẽ, tỷ lệ hoàn thành và chi phí trung bình trên các sân thành công; sân thứ ba có vách kín nên cả hai phải báo no_path. Chi phí giây được tính từ FORWARD_S và TURN_S, không phải số đo thời gian robot. Giảm EXPANSION_LIMIT để kiểm tra báo hết ngân sách; giữ cả kết quả này trong báo cáo rồi mở rộng thêm bản đồ.

Bài tập áp dụng

Có ba sân: A hoàn thành hai sân với 10 s và 12 s, sân còn lại timeout ở 30 s. B hoàn thành cả ba với 15 s, 16 s, 17 s. Tính tỷ lệ hoàn thành, thời gian trung bình trên các lượt thành công và thời gian trung bình nếu timeout được tính là 30 s. Viết kết luận không che giấu giới hạn.

Gợi ý đối chiếu sau khi tự làm

A hoàn thành 2/3 ≈ 66,7%; B 3/3 = 100%. Trung bình chỉ lượt thành công: A = 11 s, B = 16 s. Nếu tính timeout 30 s vào mọi lượt: A = 52/3 ≈ 17,33 s, B = 16 s. A nhanh ở các sân đã qua nhưng kém tin cậy trên bộ thử; cách tính khác nhau dẫn đến chỉ số khác nhau nên phải công bố rõ.

Tự kiểm tra mức hiểu

1. Bảng so sánh giữ cùng điều kiện và công bố thông tin A/B được phép dùng.

2. Tính đúng tỷ lệ thành công với đầy đủ lượt thất bại.

3. Có ít nhất một sân chưa dùng để điều chỉnh và kết luận không vượt quá bằng chứng.

Câu hỏi trắc nghiệm

Có nên bỏ các lượt không tới đích khỏi báo cáo?

1. Có

2. Không

3. Chỉ giữ lượt nhanh

Đáp án: 2. Các lượt không tới đích vẫn là bằng chứng về thuật toán. Bỏ chúng sẽ làm tỷ lệ/chi phí có vẻ tốt hơn và gây thiên lệch chọn mẫu.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEXcode VR — môi trường lập trình chính thức

https://vr.vex.com/

VEXcode VR — tài liệu Python chính thức

https://api.vex.com/vr/home/python/

VEX CS Level 1 — bài học VEXcode VR

https://education.vex.com/stemlabs/cs/cs-level-1-vexcode-vr

Python 3 — hàm có sẵn và kiểu dữ liệu

https://docs.python.org/3/library/functions.html